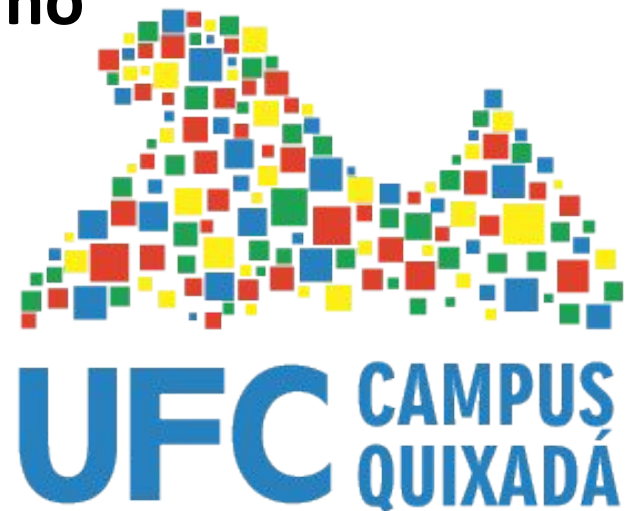


Continuous Integration (CI) Continuous Deployment (CD)

Prof. Sidartha Carvalho



CI/CD: Introdução

- CI (*Continuous Integration*)
 - Integração frequente de código em um repositório compartilhado. Cada *commit* dispara uma pipeline.
- CD (*Continuous Delivery*)
 - Preparação do código para implantação contínua em produção, mas precisa de aprovação manual.
- CD (*Continuous Deployment*)
 - Implantação automática em produção após passar em todos os testes. Entra em produção automaticamente!
- O CI/CD está relacionado ao DevOps e à entrega ágil, permitindo menor tempo de entrega e mais confiabilidade.

.

CI/CD: Introdução

- O CI/CD (Integração Contínua e Entrega/Implantação Contínua)
 - tem um papel fundamental no contexto do DevOps e da entrega ágil, proporcionando inúmeros benefícios para as equipes de desenvolvimento.
- DevOps
 - É uma cultura e conjunto de práticas que unem as equipes de desenvolvimento e operações, com o objetivo de entregar software de forma mais rápida e confiável.
 - O CI/CD é uma das principais ferramentas utilizadas para alcançar esses objetivos, automatizando e otimizando os processos.
- Entrega Ágil
 - A entrega ágil é uma metodologia que busca entregar valor ao cliente de forma incremental e iterativa, priorizando a adaptabilidade às mudanças.
 - O CI/CD alinha-se perfeitamente com essa abordagem, permitindo que novas funcionalidades sejam entregues com frequência e menor risco.

CI/CD: Introdução

- Automação
 - O CI/CD automatiza diversas etapas do processo de desenvolvimento, desde a integração do código até a implantação em produção. Isso reduz o trabalho manual, diminui a chance de erros e acelera o tempo de entrega.
- *Feedback* rápido
 - Com a automatização dos testes e da implantação, as equipes recebem feedback rápido sobre as mudanças realizadas, permitindo identificar e corrigir problemas de forma mais eficiente.
- Qualidade
 - A integração contínua garante que o código seja testado frequentemente, o que ajuda a manter a qualidade do software e a reduzir o número de bugs.

CI/CD: Introdução

- Confiabilidade
 - A implantação contínua permite que as novas versões do software sejam liberadas para produção de forma mais segura e controlada, reduzindo o risco de falhas.
 - Menor tempo de entrega
 - Ao automatizar e otimizar os processos, o CI/CD permite que as equipes entreguem novas funcionalidades com muito mais rapidez, aumentando a satisfação dos clientes.
- ❑ Em resumo, CI/CD é um dos pilares do DevOps e da entrega ágil.
- ❑ Ao automatizar e otimizar os processos de desenvolvimento, permite que as equipes entreguem software de alta qualidade com mais frequência e menor risco.
- ❑ Essa combinação poderosa traz inúmeros benefícios para as empresas, como maior competitividade, redução de custos e aumento da satisfação dos clientes.

Por que usar CI/CD?

- Problemas na abordagem tradicional
 - Integrações manuais, longas e propensas a erros.
 - Bugs identificados apenas em produção.
 - Processos de entrega demorados e inseguros.
- Benefícios do CI/CD
 - Integração frequente para detectar problemas cedo.
 - Automação de testes e *deploys*.
 - Entregas rápidas e confiáveis, com menor risco.

CI/CD: Introdução

Google: A Máquina de Atualizações

- **Milhares de lançamentos por dia**
 - O Google realiza milhares de pequenos lançamentos por dia em seus diversos produtos.
 - Essa frequência incrível é possível graças a uma infraestrutura robusta de CI/CD que automatiza todo o processo, desde a codificação até a implantação.
- **Testes extensivos**
 - Antes de cada lançamento, o código do Google passa por uma bateria de testes automatizados para garantir que não haja regressões e que a nova funcionalidade funcione como esperado.
- **Canary Releases**
 - O Google utiliza a técnica de canary releases para liberar novas funcionalidades para um pequeno grupo de usuários antes de fazer um lançamento completo.
 - Isso permite identificar e corrigir problemas rapidamente antes que afetem uma base de usuários maior.
 - “Canary releases are named after the practice of using canaries in coal mines to detect toxic gas before it could harm miners.”

CI/CD: Exemplo prático

- Imagine um cenário em que o Banco do Brasil ou outro provedor de serviços precisa implementar uma nova funcionalidade em seu aplicativo de internet banking, como a autenticação por biometria facial.
- Como você acha que deveria ocorrer essa atualização?
 - Para todos os usuários de uma vez?
 - Como deveriam ser os testes antes da disponibilização?
 - Como avaliar que não irão ocorrer problemas com a segurança dos dispositivos?

CI/CD: Exemplo prático

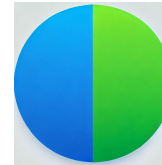
- O pipeline de CI/CD neste caso poderia seguir os seguintes passos
 - Desenvolvimento e Commit
 - Os desenvolvedores escrevem o código da nova funcionalidade e fazem commit das alterações no repositório de código-fonte.
 - Integração Contínua (CI)
 - Build: O sistema de CI (como Jenkins, GitLab CI/CD) detecta a nova alteração e inicia automaticamente um processo de build.
 - O código é compilado, empacotado e gerado um artefato (por exemplo, um arquivo WAR, JAR, APK, EXE, etc).
 - Testes Unitários
 - O artefato gerado é submetido a uma série de testes unitários automatizados para verificar se as novas funcionalidades funcionam corretamente e se não introduziram novos bugs.

CI/CD: Exemplo prático

- Na sequência...
 - Integração Contínua (CI):
 - Testes de Integração
 - Os testes de integração verificam se a nova funcionalidade interage corretamente com outras partes do sistema.
 - Análise de Código
 - Ferramentas de análise de código estática são utilizadas para identificar possíveis problemas de segurança, desempenho ou qualidade do código.

CI/CD: Exemplo prático

- Entrega Contínua (CD):



O Blue-green deployment é uma estratégia para liberar atualizações de software que envolve a criação de dois ambientes idênticos. Um ambiente (azul) executa a versão atual da aplicação, enquanto o outro (verde) executa a nova versão.

- Deploy para Ambiente de Teste
 - O artefato aprovado nos testes é automaticamente implantado em um ambiente de teste que simula o ambiente de produção.
- Testes de Aceitação
 - São realizados testes de aceitação para verificar se a nova funcionalidade atende aos requisitos de negócio e se a experiência do usuário é satisfatória.
- Deploy para Produção
 - Após a aprovação nos testes, o artefato é promovido para o ambiente de produção, utilizando técnicas como *blue-green deployment* ou *canary releases* para minimizar o risco de interrupções no serviço.

CI/CD: Exemplo prático

- Benefícios do CI/CD
 - Aumento da velocidade de entrega
 - Novas funcionalidades são entregues aos clientes de forma mais rápida, aumentando a competitividade do banco.
 - Melhoria da qualidade do software
 - A automatização dos testes garante que o software seja mais robusto e confiável.

CI/CD: Exemplo prático

- Benefícios do CI/CD
 - Redução de erros
 - A detecção precoce de problemas permite corrigir bugs de forma mais eficiente.
 - Maior agilidade na resolução de incidentes
 - A automatização do processo de *build* e *deploy* facilita a resolução de problemas em produção.
 - Melhor colaboração entre as equipes
 - O CI/CD promove uma maior colaboração entre as equipes de desenvolvimento e operações.

CI/CD: Exemplo prático

- **Desafios**

- Complexidade

- A infraestrutura do nosso exemplo (Banco do Brasil) é complexa, o que exige uma configuração robusta do pipeline de CI/CD.

- Segurança

- É fundamental garantir a segurança dos dados e sistemas durante o processo de CI/CD.

- Legislação

- As instituições financeiras estão sujeitas a uma série de regulamentações que precisam ser consideradas na implementação do CI/CD.

Componentes no CI / CD

CI/CD: Componentes

- **Pipeline CI/CD**
 - Linha de produção automatizada para software.
 - É uma sequência ordenada de etapas que um código percorre desde o momento em que é escrito até ser disponibilizado para os usuários.
- Em um pipeline de CI/CD temos
 - **Codificação:** Os desenvolvedores escrevem o código e fazem *commit* das alterações.
 - **Build:** O código é compilado e transformado em um artefato executável.
 - **Teste:** O artefato é submetido a diversos testes (unitários, de integração, etc) para garantir a qualidade.
 - **Implantação:** O artefato aprovado é implantado em um ambiente, seja ele de desenvolvimento, teste ou produção.

CI/CD: Componentes

- Imagine um pipeline como uma esteira em uma fábrica
 - Código
 - A matéria-prima que entra na fábrica.
 - Build
 - A transformação da matéria-prima em um produto.
 - Teste
 - A inspeção do produto para garantir a qualidade.
 - Implantação
 - O envio do produto final para o mercado.

CI/CD: Componentes

- Ferramentas comuns



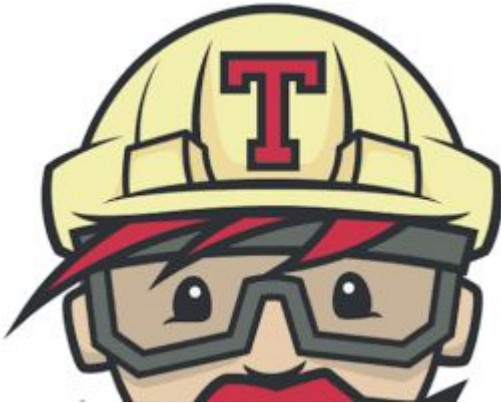
Jenkins



GitHub Actions



GitLab CI/CD



Travis CI



CI/CD: Exemplo de Pipeline

```
name: CI/CD Pipeline
on:
  push:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Run Tests
        run: npm test
      - name: Deploy to Staging
        run: echo "Deploying to staging!"
```



GitHub Actions

Staging (ou homologação) é uma área entre as etapas de desenvolvimento e produção para avaliar o produto com as características de produção, mas sem os riscos dos usuários.

CI/CD: Boas práticas

- Faça commits pequenos e frequentes.
- Automatize testes unitários, de integração e de regressão.
- Use *logs* e monitoramento para detectar problemas rapidamente.
- Planeje e teste estratégias de *rollback*.





- **Ignorar Testes Automatizados**
- Os desenvolvedores pulam a criação ou execução de testes automatizados antes de integrar ou implantar o código.
- Isso pode levar a:
 - Bugs graves que podem passar despercebidos até chegarem em produção.
 - Retrabalho: É mais difícil corrigir problemas detectados tarde no processo.
 - Perda de confiança no *pipeline*, já que ele não garante a qualidade do código.
- Exemplo Real:
 - Uma atualização de *e-commerce* que implementa descontos sem testar corretamente pode causar cálculos errados de preços e impactar as vendas.



- **Acionar Pipelines Manualmente**
- Pipelines de *build* e *deploy* são acionados manualmente, dependendo do julgamento humano.
- Consequências:
 - Processos inconsistentes: pode-se esquecer de executar testes ou criar builds específicos.
 - Maior risco de erro humano, como esquecer de incluir uma etapa importante.
 - Tempo perdido: desenvolvedores precisam intervir em etapas que poderiam ser automatizadas.
- Um desenvolvedor esquece de rodar os testes ou de incluir uma biblioteca na compilação, resultando em um erro crítico no site do cliente em produção.



- **Falta de Integração Frequente**
- Desenvolvedores trabalham em grandes blocos de código por muito tempo sem integrar com o código principal.
- Consequências:
 - Integração demorada e sujeita a conflitos, especialmente em equipes grandes.
 - Dificuldade em rastrear *bugs*, já que a mudança afeta muitas áreas do código.
 - Retrabalho: É preciso corrigir conflitos de integração antes mesmo de testar a funcionalidade.
- Um time lança um recurso após semanas de desenvolvimento e descobre que ele entra em conflito com outras alterações já implantadas.



- ***Deploys Manuais e Não Reprodutíveis***
- As etapas de implantação (*deploy*) são realizadas manualmente por scripts locais ou por pessoas diferentes.
- Consequências:
 - Inconsistência entre ambientes (*staging*, produção, etc).
 - Dificuldade para auditar quem fez o quê e quando.
 - Atrasos na entrega, especialmente em casos de dependência de um único especialista.
- Um engenheiro realiza o *deploy* de uma nova versão em produção, mas esquece de atualizar uma variável de ambiente crítica, causando falhas.



- **Ausência de Rollbacks Planejados**
- O processo de *rollback* não é planejado ou automatizado.
- Consequências:
 - Em caso de falha, a reversão para uma versão estável é difícil e demorada.
 - *Downtime* mais longo para os usuários.
 - Perda de receita e credibilidade da aplicação.
- Uma atualização com *bug* é implantada, mas a equipe demora horas para restaurar a versão anterior manualmente.



- **Falta de Monitoramento e *Feedback***
- O *pipeline* não é monitorado adequadamente e não fornece *feedback* aos desenvolvedores.
- Consequências:
 - Desenvolvedores não sabem quando algo falha, resultando em atrasos para corrigir problemas.
 - Dificuldade em diagnosticar a origem de falhas, especialmente em *pipelines* complexos.
- Um teste automatizado falha, mas ninguém percebe porque o *pipeline* não notifica a equipe.

Definições

Github Actions

<https://docs.github.com/en/actions>

CI/CD: Github Actions

- Com o Github Actions é possível automatizar, customizar e executar pipelines de desenvolvimento dentro do próprio Github.
- GitHub Actions é uma plataforma de CI/CD que permite automatizar o *build*, testes e *deploy*.
- Você pode criar *workflows* que realizam *build* e *test* a cada *pull* no seu repositório ou ao evento que desejar.

CI/CD: Github Actions

- Você pode configurar um *workflow* do GitHub Actions para ser acionado quando um evento ocorrer em seu repositório, como a abertura de um *pull request* ou a criação de uma *issue*.
- O seu workflow contém um ou mais *jobs*, que podem ser executados em ordem sequencial ou em paralelo.
- Cada *job* será executado dentro de sua própria máquina virtual ou dentro de um contêiner.

CI/CD: Github Actions

- Um *workflow* é um processo automatizado configurável que executará um ou mais *jobs*.
- Os *workflows* são definidos por um arquivo YAML que é adicionado ao seu repositório e serão executados quando acionados por um evento no repositório, manualmente ou em um horário agendado.
- Os *workflows* são definidos no diretório ``.github/workflows`` dentro do seu repositório. Um repositório pode conter vários *workflows*, cada um podendo executar um conjunto diferente de tarefas, como:
 - Compilar e testar *pull requests*.
 - Implantar sua aplicação toda vez que uma nova release for criada.
 - Adicionar uma label sempre que uma nova *issue* for aberta.

CI/CD: Github Actions

- Um **job** é um conjunto de etapas em um *workflow* que é executado no mesmo *runner* (máquina).
- Cada etapa pode ser um *script shell* ou uma ação que será executada.
- As etapas são executadas em ordem e dependem umas das outras.
- Como todas as etapas são executadas no mesmo *runner*, é possível compartilhar dados de uma etapa para outra.
- Por exemplo, você pode ter uma etapa que compila sua aplicação, seguida por uma etapa que testa a aplicação compilada.

CI/CD: Github Actions

- Você pode configurar as dependências de um *job* com outros *jobs*; por padrão, os *jobs* não possuem dependências e são executados em paralelo.
- Quando um *job* depende de outro, ele espera o *job* dependente ser concluído antes de ser executado.
- Por exemplo, você pode configurar múltiplos *jobs* de *build* para diferentes arquiteturas sem dependências entre eles, e um *job* de empacotamento que dependa desses *builds*.
- Os *jobs* de *build* serão executados em paralelo e, assim que forem concluídos com sucesso, o *job* de empacotamento será executado.

CI/CD: Github Actions

- Uma *action* é uma aplicação personalizada para a plataforma GitHub Actions que executa uma tarefa complexa, mas frequentemente repetida.
- Use uma *action* para ajudar a reduzir a quantidade de código repetitivo que você escreve nos arquivos de seus *workflows*.
- Uma *action* pode, por exemplo, clonar seu repositório do GitHub, configurar a ferramenta apropriada para seu ambiente de *build* ou configurar a autenticação com seu provedor de nuvem.
- Você pode criar suas próprias *actions* ou encontrar *actions* para usar em seus workflows no GitHub Marketplace.

CI/CD: Github Actions

- Um *runner* é um servidor que executa seus *workflows* quando eles são acionados.
- Cada *runner* pode executar apenas um *job* por vez.
- O GitHub oferece runners com sistemas operacionais Ubuntu Linux, Microsoft Windows e macOS para rodar seus *workflows*.
- Cada execução de *workflow* ocorre em uma máquina virtual nova e recém-provisionada.
- O GitHub também disponibiliza *runners* maiores, que estão disponíveis em configurações mais robustas.

CI/CD: Github Actions - Resumo

- **Workflow**
 - Arquivo YAML que descreve toda a automação — o pipeline completo. Define *o que, como e quando* algo será executado.
- **Job**
 - Uma parte do workflow.
 - Cada job executa uma tarefa maior (build, test, deploy) e pode rodar em paralelo ou em sequência.
- **Step**
 - Comandos individuais dentro de um job (ex.: `npm install`, `npm test`). Executados na ordem definida.
- **Action**
 - Passo reutilizável já pronto (ex.: `actions/checkout`) que encapsula uma funcionalidade específica.
- **Runner**
 - Máquina onde os jobs rodam (pode ser do GitHub ou própria).
- **Event**
 - Gatilho que inicia o workflow (push, pull request, release etc.).

Pipeline

Github Actions

<https://docs.github.com/en/actions>

CI/CD: Github Actions

- Para que o GitHub identifique quaisquer workflows do GitHub Actions em seu repositório, você deve salvar os arquivos de workflow em um diretório chamado `.github/workflows`. Por exemplo, `github-actions-demo.yml`.
- Você pode dar ao arquivo de workflow qualquer nome que desejar, mas deve usar `.yml` ou `.yaml` como extensão do nome do arquivo.
- YAML é uma linguagem de marcação comumente usada para arquivos de configuração.

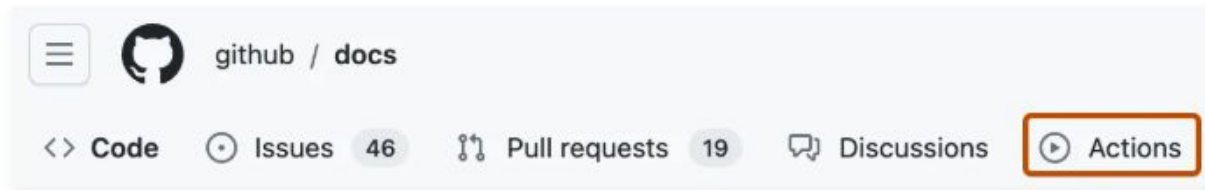
CI/CD: Github Actions

- Exemplo github-actions-demo.yml

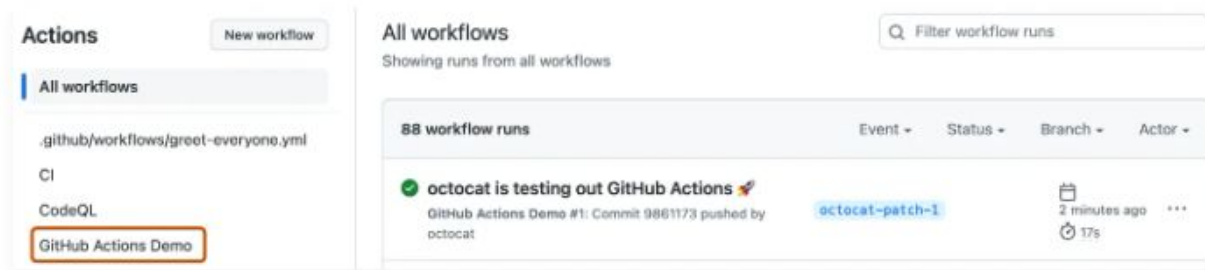
```
name: GitHub Actions Demo
run-name: ${ github.actor } is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${ github.event_name } event."
      - run: echo "🐧 This job is now running on a ${ runner.os } server hosted by GitHub!"
      - run: echo "🔍 The name of your branch is ${ github.ref } and your repository is ${ github.repository }."
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${ github.repository } repository has been cloned to the runner."
      - run: echo "💻 The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          ls ${ github.workspace }
      - run: echo "🍏 This job's status is ${ job.status }."
```

CI/CD: Github Actions

- 2 Under your repository name, click  **Actions**.

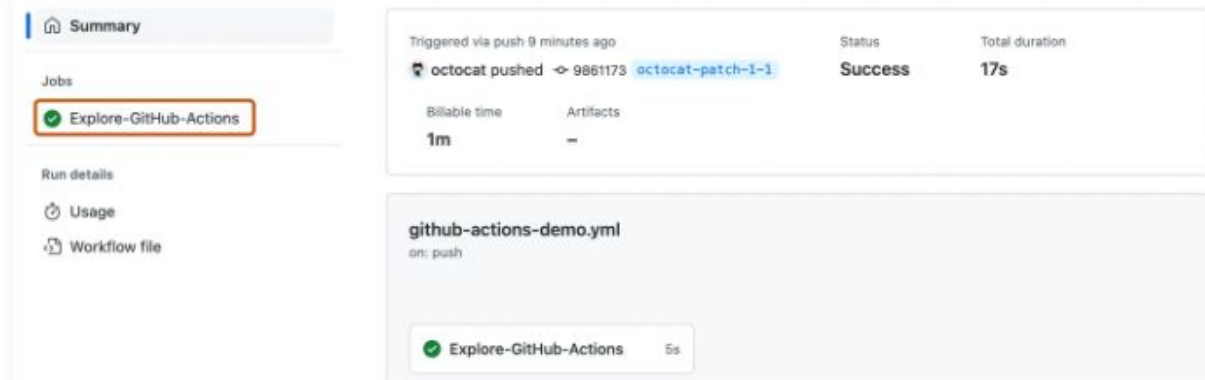


- 3 In the left sidebar, click the workflow you want to display, in this example "GitHub Actions Demo."



- 4 From the list of workflow runs, click the name of the run you want to see, in this example "USERNAME is testing out GitHub Actions."

- 5 In the left sidebar of the workflow run page, under **Jobs**, click the **Explore-GitHub-Actions** job.



CI/CD: Github Actions

Explore-GitHub-Actions

succeeded 13 minutes ago in 5s

🔍 Search logs



- > Set up job 1s
- > Run echo "🔔 The job was automatically triggered by a push event." 0s
- > Run echo "👤 This job is now running on a Linux server hosted by GitHub!" 0s
- > Run echo "🔍 The name of your branch is refs/heads/octocat-patch-1 and your repository is oct... 0s
- > Check out repository code 1s
- > Run echo "💡 The octo-org/octo-repo repository has been cloned to the runner." 0s
- > Run echo "💻 The workflow is now ready to test your code on the runner." 0s
- > List files in the repository 0s
- > Run echo "🟢 This job's status is success." 0s
- > Post Check out repository code 0s
- > Complete job 0s

CI/CD: Github Actions

- Principais comandos
 - name
 - Define o nome do workflow.
 - on
 - Define os eventos que disparam o workflow (por exemplo, push, pull_request).
 - jobs
 - Define os trabalhos do workflow. Cada trabalho contém etapas.

```
name: CI/CD Workflow
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout Code
```

```
        uses: actions/checkout@v3
```

CI/CD: Github Actions

- Eventos de Gatilho (para ser usado com o on)
 - push
 - Dispara o workflow quando há commits em uma branch específica.
 - pull_request
 - Executa o workflow em PRs abertos, sincronizados ou reabertos.
 - workflow_dispatch
 - Permite disparar o workflow manualmente.
 - schedule
 - Define execuções agendadas (usando cron [tempo]).

CI/CD: Github Actions

Exemplos de uso

on:

push:

branches:

- main

on:

pull_request:

on:

workflow_dispatch:

on:

schedule:

- cron: "0 0 * * *" # Executa diariamente à meia-noite

- Configurações de Jobs
 - runs-on
 - Especifica o ambiente onde o job será executado.
 - steps
 - Define as etapas dentro de um job.
 - needs
 - Configura dependências entre jobs.

CI/CD: Github Actions

```
runs-on: ubuntu-latest
```

```
steps:
```

- name: Checkout Code
uses: actions/checkout@v3
- name: Run Tests
run: npm test

```
jobs:
```

```
build:
```

```
runs-on: ubuntu-latest
```

```
test:
```

```
runs-on: ubuntu-latest
```

```
needs: build
```

CI/CD: Github Actions

- Ações Reutilizáveis
 - uses
 - Especifica uma ação pronta para uso (por exemplo, do marketplace).
 - run
 - Executa comandos shell diretamente no ambiente.
 - with
 - Passa parâmetros para uma ação.

```
- uses: actions/checkout@v3
```

```
- run: npm install
```

```
- uses: actions/setup-node@v3  
  with:  
    node-version: 16
```

CI/CD: Github Actions

- **env**
 - Define variáveis de ambiente.
- **secrets**
 - Utiliza valores seguros armazenados no repositório
- **if**
 - Executa um passo condicionalmente.

env:

NODE_ENV: production

- **run:** echo `${{ secrets.MY_SECRET }}`

- **run:** echo "Running step"

if: github.event_name == 'push'

CI/CD: Github Actions

- actions/upload-artifact
 - Salva artefatos gerados no workflow.
- actions/download-artifact
 - Faz download de artefatos de um job anterior.
 - Uma forma de compartilhar arquivos entre jobs da mesma execução.

```
- name: Upload Build
  uses: actions/upload-artifact@v3
  with:
    name: build-artifact
    path: build/

- name: Download Build
  uses: actions/download-artifact@v3
  with:
    name: build-artifact
```


CI/CD: Github Actions

- debug
 - Habilita logs detalhados.
- continue-on-error
 - Permite que o workflow continue mesmo após erros.

```
- run: echo "DEBUG: ${ steps.build.outputs.result }"
```

```
- run: npm test  
  continue-on-error: true
```

CI/CD: Github Actions

```
name: Node.js CI/CD Pipeline
on:
  push:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 16
      - run: npm install
      - run: npm test
      - name: Upload Artifact
        uses: actions/upload-artifact@v3
        with:
          name: test-results
          path: ./test-results/
```

Dúvidas?



www.shutterstock.com · 744867163